T2. 串口封装

📄 任务要求

实现串口的收发消息（需要优雅）

"Any fool can write code that a computer can understand. Good programmers write code that humans can understand." — *Martin Fowler*

“任何傻瓜都能写出机器能懂的代码。优秀的程序员写出的是人类能懂的代码。”— 面向对象之父

🥉 青铜段位：初窥门径 —— 阻塞式控制

- **【核心目标】** 确认物理链路（打通逻辑）。
- **【教学思维】** **线性思维**。理解“指令”是按顺序执行的，意识到 CPU 会因为等待硬件而“浪费时间”。
- **【代码特征】** 直接调用 `HAL_UART_Transmit`。

```
/* 逻辑：CPU 必须等数据发完才能执行下一行 */
uint8_t msg[] = "System Online\r\n";
HAL_UART_Transmit(&huart1, msg, sizeof(msg), 100);
// 痛点：发送 100 字节在 115200 波特率下约耗时 8.6ms，这对 1ms 频率的控制环路是致命的。
```

🥈 白银段位：小试牛刀 —— 基础异步中断

- **【核心目标】** 引入“异步”概念。
- **【教学思维】** **事件驱动初探**。理解“打断”机制，意识到 CPU 可以在等待时做别的事。
- **【代码特征】** 开启接收中断，在回调函数中处理单字节。

```
/* 开启中断监听 */
HAL_UART_Receive_IT(&huart1, &rx_byte, 1);

/* 回调函数：每收到 1 字节进一次中断 */
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart) {
    if(huart->Instance == USART1) {
        process_byte(rx_byte); // 处理
        HAL_UART_Receive_IT(&huart1, &rx_byte, 1); // 重新开启
    }
```

```
}
// 痛点：中断过于频繁（每秒上万次），系统开销大；变量散乱。
```

🥇 黄金段位：渐入佳境 —— DMA 与高效搬运

- **【核心目标】** 极致压榨硬件性能。
- **【教学思维】** **资源调度思维**。理解 DMA（直接存储器访问）是“专业搬运工”，让 CPU 彻底从字节搬运中解放。
- **【代码特征】** DMA 接收 + 串口空闲中断（IDLE）。

```
/* 逻辑：只有一帧数据收完了，CPU 才介入处理一次 */
HAL_UART_Receive_DMA(&huart1, rx_buffer, 128); // 硬件自动搬运

void USART1_IRQHandler(void) {
    if (__HAL_UART_GET_FLAG(&huart1, UART_FLAG_IDLE)) {
        __HAL_UART_CLEAR_IDLEFLAG(&huart1);
        uint16_t len = 128 - __HAL_DMA_GET_COUNTER(huart1.hdmarx);
        Process_Frame(rx_buffer, len); // 整包处理
    }
}
```

🍀 铂金段位：登堂入室 —— C++ 对象化封装

- **【核心目标】** 建立“对象”意识，消灭全局变量。
- **【教学思维】** **封装与自愈**。隐藏 `&huart1` 等底层句柄，引入“掉线监测”和“错误自修复”。
- **【代码特征】** 使用 C++ Class，将数据、状态、初始化方法打包。

```
class SerialPort {
private:
    UART_HandleTypeDef* m_handle;
    bool m_is_online;
    uint32_t m_last_tick;

public:
    SerialPort(UART_HandleTypeDef* h) : m_handle(h), m_is_online(false) {}
    void start();           // 开启 DMA 与空闲中断
    void check_status();    // 监控是否掉线，若 ErrorCode 报错则自动重启
    bool is_alive() { return m_is_online; }
```

```
};
/* 业务层不再看得到 HAL 函数，只有 vision_port.start() */
```

💎 钻石段位：超凡入圣 —— Lambda 钩子与解耦

- **【核心目标】** 架构层面的绝对解耦（开闭原则）。
- **【教学思维】** **契约思维（Hook）**。底层驱动不关心业务逻辑，只提供“触发钩子”。
- **【代码特征】** 函数指针回调 + C++ Lambda 表达式。

```
/* --- 底层驱动：我只负责收数据，收到了就踢一下钩子 --- */
using Callback_t = void(*)(uint8_t* data, uint16_t len);
void register_callback(Callback_t cb) { m_callback = cb; }

/* --- 业务层：我想干什么，直接挂在钩子上 --- */
vision_port.register_callback([](uint8_t* data, uint16_t len) {
    if (data[0] == 0xA5) {
        // 直接在这里写解析逻辑，或者调用算法模块
        update_target_angle(data);
    }
});
// 优势：驱动类代码永远不需要修改，新功能只需通过 Lambda 动态“挂载”。
```

👑 王者段位：分布式通讯总线 (The Message Bus)

【核心目标】 彻底抹除模块间的物理依赖。

【教学思维】 **解耦的极致**。云台算法不需要知道数据是从 `huart1` 还是 `huart6` 来的，它只订阅“视觉目标”这个主题。

【代码特征】 模板类、单例模式、匿名函数、主题订阅。

---

1. 核心骨架：通讯总线类 (`MessageBus.hpp`)

这是系统的“心脏”。它维护了一个名单，记录了谁对什么数据感兴趣。

```
#include <vector>
#include <functional>
```

```cpp
#include <map>
#include <string>

// 王者段位的精髓：万物皆可为 Topic
template <typename T>
class MessageBus {
public:
    // 定义订阅者类型：一个处理特定数据结构的函数
    using Subscriber = std::function<void(const T&)>;

    // 单例模式，保证全局只有一个总线
    static MessageBus<T>& Instance() {
        static MessageBus<T> instance;
        return instance;
    }

    // 订阅：我要听这个消息
    void subscribe(Subscriber sub) {
        subscribers.push_back(sub);
    }

    // 发布：广播这个消息
    void publish(const T& msg) {
        for (auto& sub : subscribers) {
            sub(msg); // 通知所有订阅者
        }
    }

private:
    std::vector<Subscriber> subscribers;
    MessageBus() = default;
};
```

---

2. 底层驱动：只管广播（`Vision_Driver.cpp`）

底层驱动变得异常纯粹：它只负责收包、解包，然后把解好的结构体扔到总线上，从此“两耳不闻窗外事”。

```cpp
// 假设这是我们的视觉数据结构
struct VisionData {
    float yaw;
    float pitch;
    bool target_found;
```

```
};

// 串口中断回调
void HAL_UART_IdleCallback(uint8_t* raw_data, uint16_t len) {
    VisionData decoded_data;
    // ... 这里进行复杂的 DMA 数据解包和校验 ...

    // 【王者之举】: 直接发布到总线，驱动层完全不知道谁会用到这些数据
    MessageBus<VisionData>::Instance().publish(decoded_data);
}
```

---

3. 业务层：自由订阅（`Gimbal_Task.cpp`）

业务层（如云台控制）不再需要包含任何串口的头文件，它只需要关注数据本身。

```
void Gimbal_Init() {
    // 云台任务只需要订阅“视觉数据”这个主题
    MessageBus<VisionData>::Instance().subscribe([](const VisionData& msg) {
        if (msg.target_found) {
            // 这里直接控制云台，代码逻辑极其清晰
            Gimbal_Control(msg.yaw, msg.pitch);
        }
    });
}
```